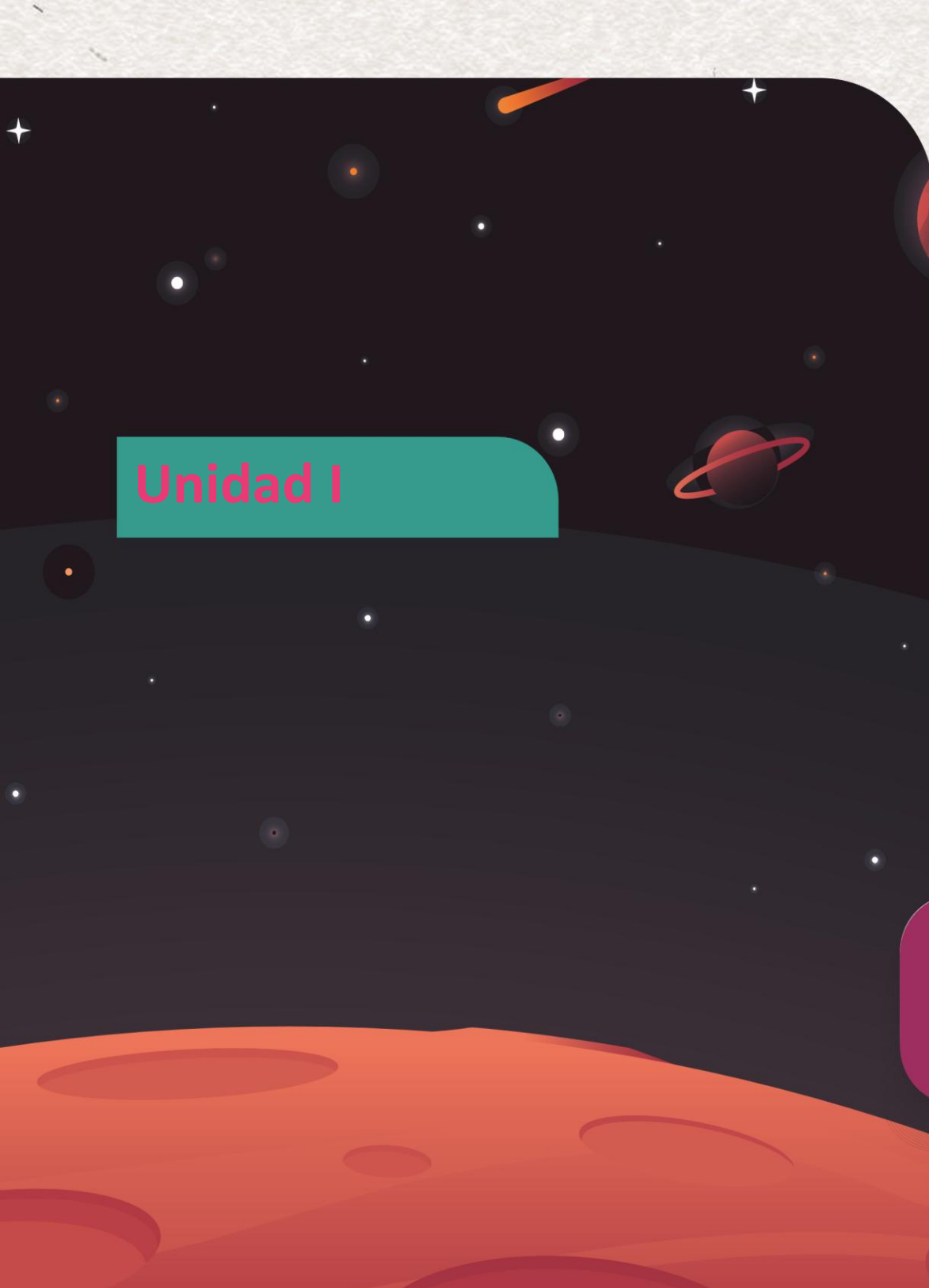




CESDE

Santiago Yosa González

Ingeniero Sistemas de Información



Unidad I

Funciones en JavaScript:

Funciones regulares

Parámetros

Retorno de valores.

Estructuración del sitio Web

Situación

Imagina que eres un chef y tienes un libro de recetas. En las sesiones pasadas, si querías hacer un pastel (ej. nuestro ejercicio "FizzBuzz"), tenías que escribir la receta completa en un papel, usarla y luego botar el papel. Si querías hacer *otro* pastel, tenías que volver a escribir la receta desde cero.

Una **función** es como escribir esa receta *una sola vez* y guardarla en tu libro de recetas con un nombre (ej. "Pastel de Chocolate").

Ahora, cada vez que quieras un pastel, no reescribes la receta; simplemente abres el libro, buscas el nombre y "llamas" a la receta. Puedes "llamarlo" 100 veces. Mejor aún, la receta puede tener "variables" (ej. "cantidad de azúcar") para que puedas hacer un pastel más o menos dulce.

Las funciones nos permiten **definir** una lógica compleja una vez y **llamarla** miles de veces, de forma simple y limpia.

¿Qué son las Funciones?

Una **función** en JavaScript es un "**bloque de código**" que agrupa una serie de instrucciones para realizar una tarea específica. Le das un nombre, y cada vez que quieres que la tarea se ejecute, simplemente "llamas" a la función por su nombre.

¿Por qué usamos funciones?

- **Reutilización de Código:** Evitas repetir el mismo código una y otra vez. Escribes la lógica una vez y la usas cuantas veces sea necesario. ¡Menos código, menos errores!
- **Modularidad:** Divides tu programa grande en piezas más pequeñas y manejables (nuestras "mini-máquinas"). Cada pieza se encarga de una tarea específica, lo que facilita entender, probar y mantener tu código.
- **Legibilidad:** Un programa con funciones bien definidas es más fácil de leer y entender, tanto para ti como para otros desarrolladores. Los nombres de las funciones nos dicen qué hacen.

Declaración y Llamada de Funciones

Para crear nuestra función necesitamos **declararla**, con un nombre. Luego, para "ejecutarla", necesitamos **llamar** o **invocar** la función.

Sintaxis Básica de Declaración (Función Regular):

```
function saludar() { // Abre la función
// Aquí va el "cuerpo" de la función: las instrucciones que realizará
console.log("¡Hola, mundo!");
console.log("¡Bienvenido a JavaScript!");
} // Cierra la función
```

Llamada o Invocación de una Función:

Para que la función haga su trabajo, tenemos que "llamarla" o "invocarla". Esto se hace simplemente escribiendo su nombre seguido de paréntesis ().

```
function saludar() {  
  console.log("¡Hola, mundo!");  
  console.log("¡Bienvenido/a a JavaScript!");  
}
```

```
saludar();
```

Hoisting de Funciones

Una particularidad de las **declaraciones de función** es el hoisting. Esto significa que JavaScript "eleva" la declaración de la función al principio de su ámbito antes de ejecutar el código. Por lo tanto, puedes llamar a una función declarada con *function* *antes* de que su definición aparezca en el código.

Parámetros y Argumentos (Entradas)

Una función que solo hace una cosa (como saludar siempre igual) no es muy útil. Necesitamos que sea flexible. Para eso, le pasamos "ingredientes".

- **Parámetros:** Son las "variables placeholder" que defines en la receta. Son los nombres de los ingredientes que la función *espera* recibir.
- **Argumentos:** Son los *valores reales* que le pasas a la función cuando la invocas.

Parámetros

Los **parámetros** son variables especiales que se declaran en los paréntesis de la función. Actúan como marcadores de posición para los valores que la función necesita recibir cuando es llamada.

Cuando llamas a la función, le pasas **argumentos**. Los argumentos son los valores reales que se asignan a los parámetros.

```
function presentarPersona(nombre, edad) {  
  console.log(`Hola, mi nombre es ${nombre} y tengo ${edad} años.`);  
}
```

```
presentarPersona("Alicia", 25);  
presentarPersona("Roberto", 35);
```


Parámetros por Defecto

A veces, queremos que un parámetro tenga un valor predefinido si no se proporciona un argumento al llamar a la función. Esto se logra con los parámetros por defecto.

```
function saludarPersonalizado(nombre = "Invitado") { // 'Invitado' es el valor por defecto
  console.log(`¡Hola, ${nombre}!`);
}
```

```
saludarPersonalizado("María"); // Salida: ¡Hola, María!
```

```
saludarPersonalizado();      // Salida: ¡Hola, Invitado! (no se pasó argumento, usa el defecto)
```

Retorno de Valores (return)

El return es la palabra clave que usamos dentro de una función para **devolver un valor** a donde la función fue llamada.

```
function calcularDoble(numero) {  
  const doble = numero * 2;  
  return doble;  
}
```

```
const resultado1 = calcularDoble(5);  
console.log(`El doble de 5 es: ${resultado1}`);
```

```
const resultado2 = calcularDoble(5);  
console.log(`El doble de 5 es: ${resultado1}`);
```


Retornar Múltiples Valores (indirectamente): Una función solo puede devolver *un* valor. Sin embargo, si necesitas devolver varias piezas de información, puedes encapsularlas en un **objeto** o un **array**.

```
function calcularOperaciones(a, b) {  
  let suma = a + b;  
  let resta = a - b;  
  let multiplicacion = a * b;  
  
  return {  
    suma: suma,  
    resta: resta,  
    multiplicacion: multiplicacion  
  };  
}
```

```
let resultados = calcularOperaciones(10, 5);  
console.log("Suma:", resultados.suma);           // 15  
console.log("Resta:", resultados.resta);          // 5  
console.log("Multiplicación:", resultados.multiplicacion); // 50
```

Ámbito (Scope) Básico de Funciones

Variables Locales: Las variables declaradas *dentro* de una función solo existen y son accesibles *dentro* de esa función. Son "locales" a esa función y no pueden ser vistas desde fuera.

El "Scope" o Ámbito es el "contexto" donde viven las variables.

Regla de "Las Vegas": Lo que se declara *dentro* de una función, se queda *dentro* de la función.

Acceso a Variables Globales: Las funciones pueden acceder a variables declaradas *fuera* de ellas (variables globales), pero no es una buena práctica depender demasiado de esto, ya que puede hacer el código más difícil de rastrear y probar.

```
let variableGlobal = "Soy global";  
function miFuncion() {  
  let variableLocal = "Soy local, solo existo aquí dentro.";  
  console.log(variableGlobal); // Sí puedo ver la variable global  
  console.log(variableLocal);  
}
```

```
miFuncion(); // Muestra: // Soy global //
```

```
Soy local, solo existo aquí dentro.  
console.log(variableGlobal); // Muestra: Soy global  
//console.log(variableLocal);
```

Ejemplo 1: Función Simple (Sin Parámetros, Sin Retorno)

Imprime un separador de texto.

```
function imprimirSeparador() {  
  console.log("=====");  
}  
console.log("Texto 1");  
imprimirSeparador();  
console.log("Texto 2");
```


Ejemplo 2: Función con Parámetros (Sin Retorno)

Toma un nombre y lo saluda.

```
function saludarPersona(nombre) {  
  console.log(`¡Bienvenido, ${nombre}!`);  
}  
saludarPersona("Carlos");  
saludarPersona("Maria");
```

Ejemplo 3: Función Completa (Parámetros y Retorno)

Calcula el área de un rectángulo.

```
function calcularArea(ancho, alto) {  
  if (ancho <= 0 || alto <= 0) {  
    return 0; // Podemos retornar temprano si la entrada es inválida  
  }  
  const area = ancho * alto;  
  return area;  
}
```

```
const areaDeMiPared = calcularArea(10, 5);  
console.log(`El área de la pared es: ${areaDeMiPared} m2`);
```

Práctica Acompañada

Práctica A "Función: Calculadora de Propinas".

Misión: "Calculadora de Propinas" en una función reutilizable que *devuelva* el total a pagar.

Paso a paso para el estudiante:

1. **Abre tu consola.**
2. **Define la función:** Escribe `function calcularTotal(montoBoleta, porcentajePropina) { ... }`.
3. **Mueve la lógica adentro:** Copia la lógica de cálculo que hicimos en la Sesión 1.
 - `const totalPropina = montoBoleta * porcentajePropina;`
 - `const totalPagar = montoBoleta + totalPropina;`
4. **Añade el return:** En lugar de `console.log`, usa `return totalPagar`; para que la función devuelva el valor.
5. **Llama (invoca) a la función:** Ahora, usa la función. Guárdala en una variable:
 - `const cuentaMesa1 = calcularTotal(80, 0.15);`
 - `const cuentaMesa2 = calcularTotal(120, 0.20);`
6. **Muestra los resultados:** `console.log(\La Mesa 1 paga: ${cuentaMesa1}\`);``

Solución

// 1. Definición de la función con parámetros

```
function calcularTotal(montoBoleta, porcentajePropina) {
```

// 2. Lógica de cálculo interna

```
const totalPropina = montoBoleta * porcentajePropina;
```

```
const totalPagar = montoBoleta + totalPropina;
```

// 3. Retorno del valor

```
return totalPagar;
```

```
}
```

// 4. Invocación y uso

```
const cuentaMesa1 = calcularTotal(80, 0.15); // Calcula para una boleta de 80 con 15%
```

```
const cuentaMesa2 = calcularTotal(120, 0.20); // Calcula para una boleta de 120 con 20%
```

// 5. Muestra de resultados

```
console.log(`Total Mesa 1: ${cuentaMesa1}`); // Total Mesa 1: $92
```

```
console.log(`Total Mesa 2: ${cuentaMesa2}`); // Total Mesa 2: $144
```

Práctica B "Función: Conversor de Temperatura"

Misión: Convertir la "Calculadora de Temperatura" de la Sesión 1 en una función reutilizable.

Paso a paso para el estudiante:

1. **Define la función:** Escribe una función llamada `convertirCelsiusAFahrenheit`.
2. **Define el parámetro:** La función debe aceptar un parámetro, llámalo `tempCelsius`.
3. **Implementa la lógica:** Dentro de la función, usa la fórmula $(C * 9/5) + 32$ para calcular los grados Fahrenheit.
4. **Retorna el valor:** Usa `return` para devolver el resultado del cálculo.
5. **Prueba tu función:** Llama a la función con diferentes valores y muéstralos en la consola.
 - `console.log(\25°C son ${convertirCelsiusAFahrenheit(25)}°F`);``
 - `console.log(\0°C son ${convertirCelsiusAFahrenheit(0)}°F`);``

Ejercicio Independiente

Misión: "El Verificador de Contraseñas Funcional"

Vamos a crear una función que valide si una contraseña es "fuerte" o no, y que nos devuelva un valor que podamos usar.

Instrucciones:

1. **Crea una función** llamada validarContrasena
2. La función debe aceptar un **parámetro**: contrasena.
3. **Dentro de la función**, usarás los condicionales (if...else if...else) para implementar las siguientes reglas:
 - **SI** la contraseña tiene **menos de 8 caracteres** (contrasena.length < 8), la función debe return el string "Error: Contraseña demasiado corta."
 - **SI NO, PERO SI** la contraseña es **exactamente** "12345678", la función debe return el string "Error: Contraseña insegura."
 - **SI NO** (para cualquier otro caso), la función debe return el string "Contraseña segura. ¡Buen trabajo!".
4. **Prueba tu función**: Llama a la función varias veces con diferentes contraseñas y guarda el resultado en variables.
 - let resultado1 = validarContrasena("123");
 - let resultado2 = validarContrasena("12345678");
 - let resultado3 = validarContrasena("passwordseguro123");
5. **Muestra los resultados** en la consola.

Criterios de Logro Esperados:

- La función validarContrasena está correctamente declarada y acepta un parámetro.
- La función utiliza return en todos los caminos lógicos (en el if, else if y else).
- El código no usa console.log *dentro* de la función, sino que retorna los valores.
- Las llamadas a la función fuera de ella capturan el valor retornado y lo imprimen.

¿Debería Saber Algo Más?

Hemos aprendido la "declaración de función" clásica (`function miFuncion() {}`). Esta es la base. El siguiente nivel de maestría en funciones incluye entender cómo JavaScript las trata.

1. Las Funciones son "Ciudadanos de Primera Clase": Esto significa que, en JavaScript, una función es solo otro **tipo de dato**, ¡igual que un Number o un String! Puedes hacer con ellas lo mismo que haces con las variables:

- **Asignarlas a variables:** `const miFuncion = function() { ... };` (Esto se llama **Expresión de Función**).
- **Pasarlas como argumentos a otras funciones:** El callback en `setTimeout` o `addEventListener`.
- **Retornarlas desde otras funciones.**

2. Expresiones de Función vs. Declaraciones:

- `function saludar() {}` (Declaración): Es "elevada" (hoisted). El navegador la "mueve" al principio de tu código antes de ejecutarlo. Puedes llamarla *antes* de definirla.
- `const saludar = function() {};` (Expresión): No es "elevada". Se comporta como una variable normal. No puedes usarla antes de declararla (Temporal Dead Zone). Esta es la forma **más predecible y recomendada** en el código moderno.

3. Arrow Functions (Funciones de Flecha) (ES6 - Avanzado): Son la sintaxis moderna y corta para escribir expresiones de función. Verás esto en el 99% del código moderno.

- **Expresión Antigua:** `const sumar = function(a, b) { return a + b; };`
- **Función de Flecha:** `const sumar = (a, b) => a + b;`
- Es solo "azúcar sintáctico" (una forma más corta) para la misma idea. Lo dominarás con la práctica.

CESDE | comfama